

Estado distribuido/Patterns

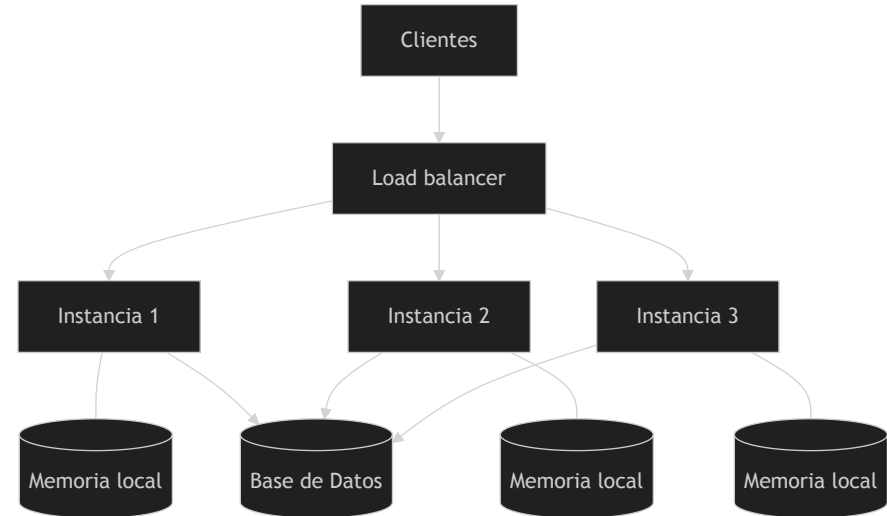
Patrones con Valkey

DOCENTES:

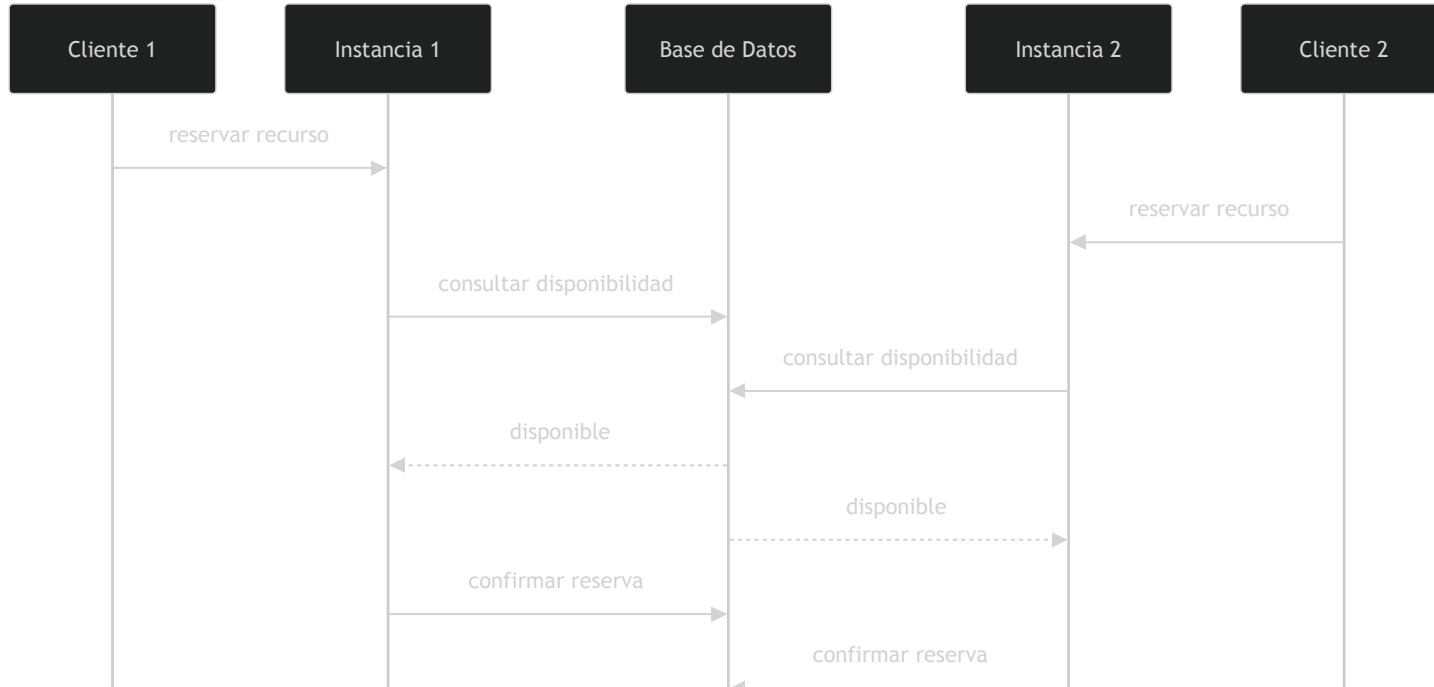
- Ing. Federico Castañeda
- Ing. Carlos Maceira
- Ing. Franco Rupnik

Arquitectura de un servicio con múltiples instancias

- El microservicio es una unidad lógica desplegada como múltiples procesos.
- Cada instancia posee memoria y ciclo de vida independientes.
- El balanceador puede dirigir cada request a una instancia diferente.
- El autoscaling, los reinicios y los despliegues reemplazan instancias continuamente.



Condiciones de carrera entre instancias



La secuencia «consultar, decidir y actualizar» debe preservar la invariante del negocio como una única operación.

Requerimientos de sincronización distribuida

Coordinación

- **Exclusión mutua:** una sola instancia ejecuta una sección crítica.
- **Control de capacidad:** como máximo N procesos usan un recurso.
- **Deduplicación:** una operación lógica se procesa una sola vez.

Estado y comunicación

- **Base de datos:** las instancias consultan una representación de datos común.
- **Propagación de cambios:** un evento se comunica a los interesados.
- **Distribución de trabajo:** cada tarea se asigna a un consumidor.

Clasificación del estado de aplicación

Tipo de estado	Ejemplos	Propiedad esperada
Local y efímero	request en curso, pool de conexiones, buffer	Puede perderse con la instancia
Compartido de coordinación	cupos, leases, claves de idempotencia	Operaciones atómicas y expiración
Durable de negocio	pedidos, pagos, inventario	Fuente de verdad y persistencia
Derivado o cacheado	rankings, consultas frecuentes, vistas	Reconstruible; admite obsolescencia controlada

Las decisiones sobre recursos compartidos no pueden depender exclusivamente del estado local de una instancia.

Valkey

Proyecto Valkey

Qué es

- Un motor de datos *in-memory*, distribuido y de código abierto.
- Nació como un **fork de Redis 7.2.4** y conservó inicialmente su protocolo, comandos y estructuras de datos.
- Se publica bajo licencia **BSD-3-Clause** y posee gobernanza neutral dentro de la Linux Foundation.

Un *fork* copia una base de código para continuar su desarrollo como un proyecto independiente.

Origen de la separación

20 de marzo de 2024 – Redis

Redis Ltd. anunció que las versiones nuevas dejarían BSD-3-Clause y usarían RSALv2 y SSPLv1 para restringir ciertos usos comerciales como servicio administrado.

28 de marzo de 2024 – Valkey

Contribuidores de Redis continuaron el último código BSD bajo un proyecto independiente de la Linux Foundation.

Desde mayo de 2025

Redis 8 volvió a ofrecer una opción *open source* mediante AGPLv3. Redis y Valkey permanecen como proyectos separados.

Fuente: <https://valkey.io/topics/history/>

Atomicidad de las operaciones

Secuencia no atómica

```
disponibles = GET recurso

if disponibles > 0:
    DECR recurso
    confirmar()
```

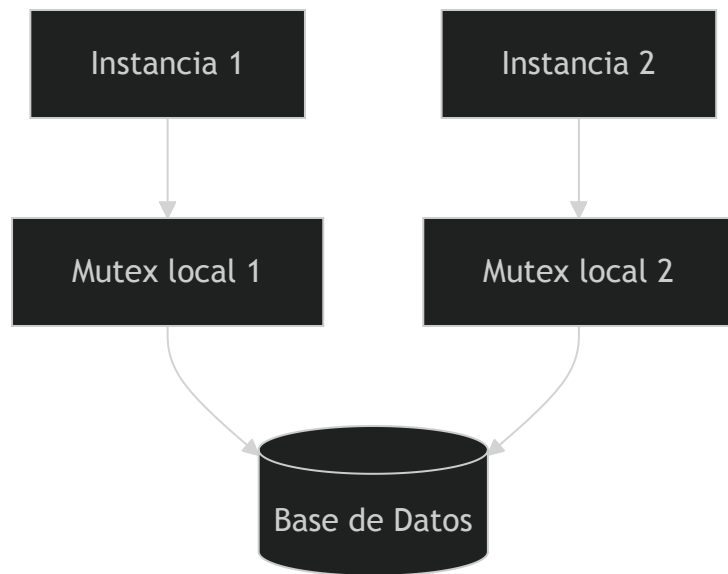
Entre la lectura y la modificación, otra instancia puede observar el mismo valor y tomar la misma decisión.

Operación atómica

- Verifica la precondition.
- Modifica el estado.
- Devuelve el resultado.
- No admite intercalaciones durante su ejecución.

Puede implementarse mediante una operación nativa, un script, una transacción o una actualización condicional en la base de datos.

Mutex local y exclusión mutua distribuida



Cada mutex serializa los threads de su propio proceso, pero ambos procesos pueden acceder simultáneamente al recurso.

Propiedad	Mutex local	Lock distribuido
Alcance	Un proceso	Varias instancias
Coordinador	Memoria local	Servicio compartido
Caída del proceso	Libera el mutex	Requiere expiración
Costo	Bajo	Red y almacenamiento

Lock con propietario y expiración

Adquisición

```
SET lock:recurso <owner-id> NX PX 30000
```

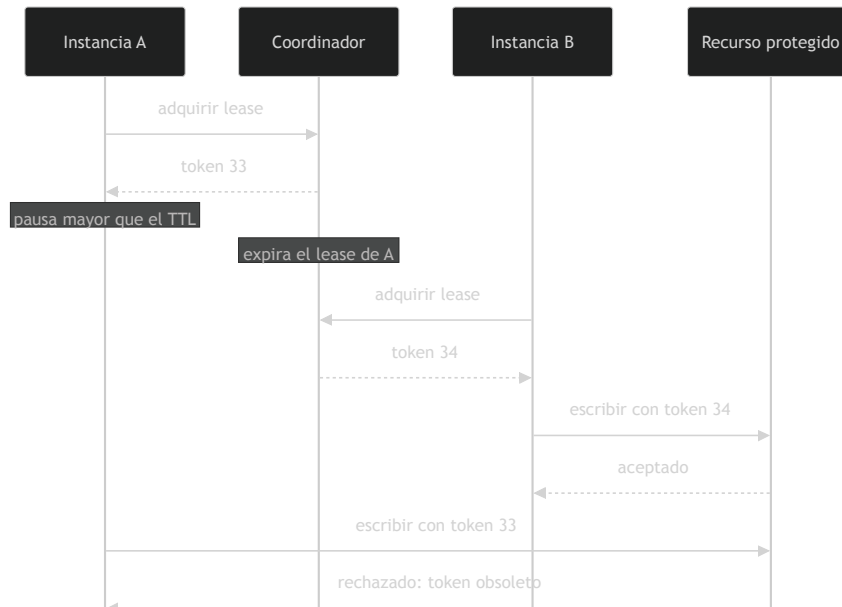
- NX: sólo crea la key si no existe.
- PX: establece la vigencia en milisegundos.
- owner-id: identifica esta adquisición.

Liberación condicional

```
if server.call("GET", KEYS[1]) == ARGV[1] then  
    return server.call("DEL", KEYS[1])  
end  
return 0
```

La comparación y el borrado se ejecutan atómicamente.

Expiración del lease y fencing tokens



- Cada adquisición recibe un token estrictamente creciente.
- El recurso conserva el mayor token observado y rechaza valores anteriores.
- El owner-id protege la liberación; el fencing token protege el recurso frente a operaciones atrasadas.

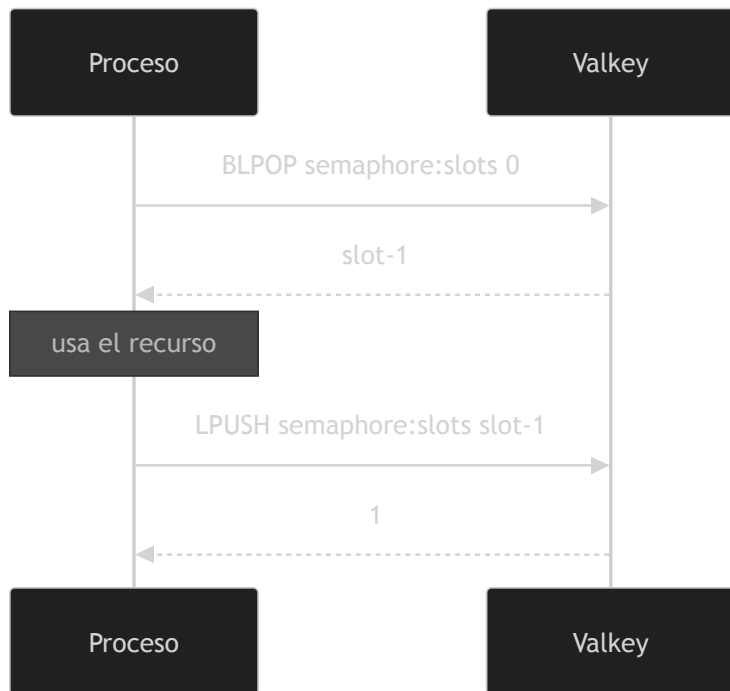
Fuente: <https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html>

Valkey – Semáforo

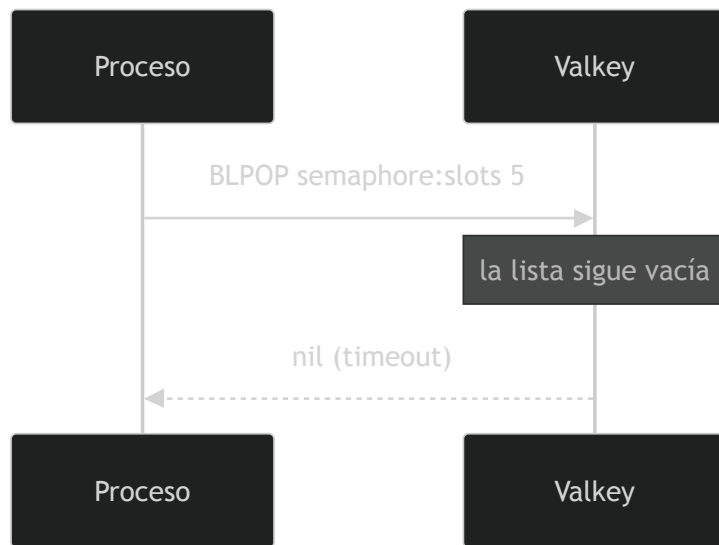
- Implementación de un semáforo distribuido.
- Se usa para controlar recursos limitados en un ambiente distribuido.
- Se emplea una lista como estructura de datos. Sus elementos pueden identificar cada recurso disponible.
- Se define una key por semáforo y los procesos ejecutan BLPOP y LPUSH a medida que adquieren o liberan recursos.

Valkey – Semáforo: adquisición y timeout

Liberación normal



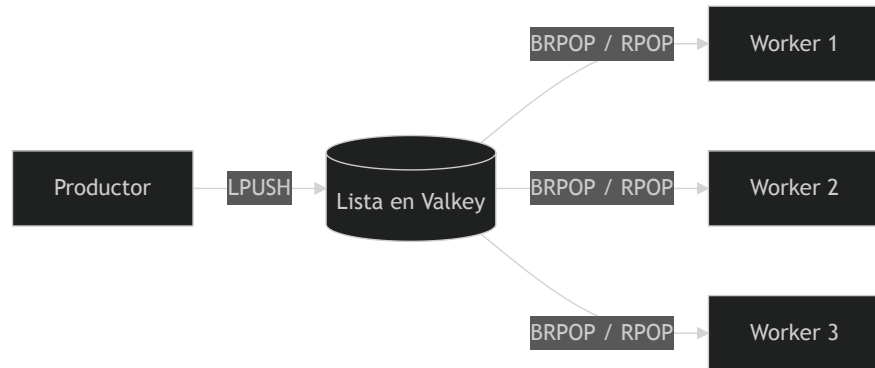
Espera con timeout



Fuente: <https://valkey.io/topics/lists/>

Valkey – Message Queue

- Implementación de una cola de mensajes.
- Una key por cola.
- Se hace LPUSH para agregar mensajes y BRPOP o RPOP para desencolarlos.
- Los destinatarios pueden bloquearse indefinidamente con timeout 0.



Fuente: <https://valkey.io/topics/lists/>

Valkey – Cache-Aside: hit y miss

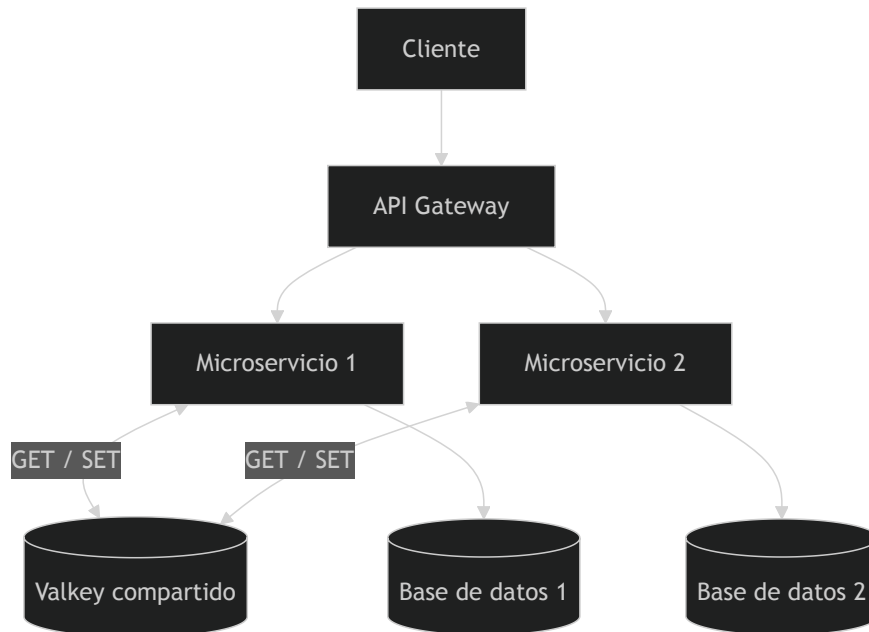
Database Cache

En búsquedas específicas o genéricas:

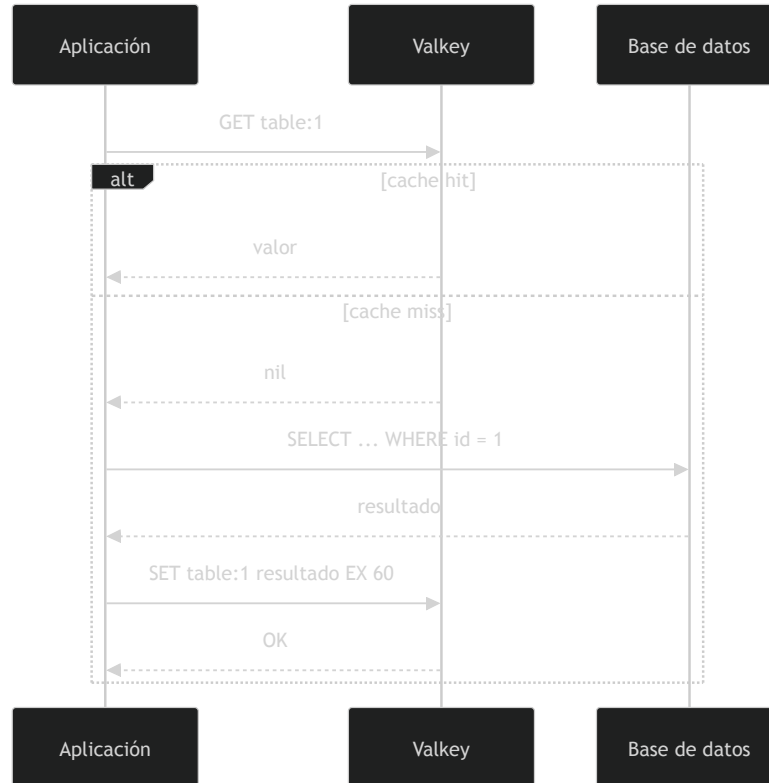
```
select *  
  from table  
 where id = 1
```

```
GET table_1
```

```
sha1("select * from table where id = 1")  
GET hash_1fde35 ...
```

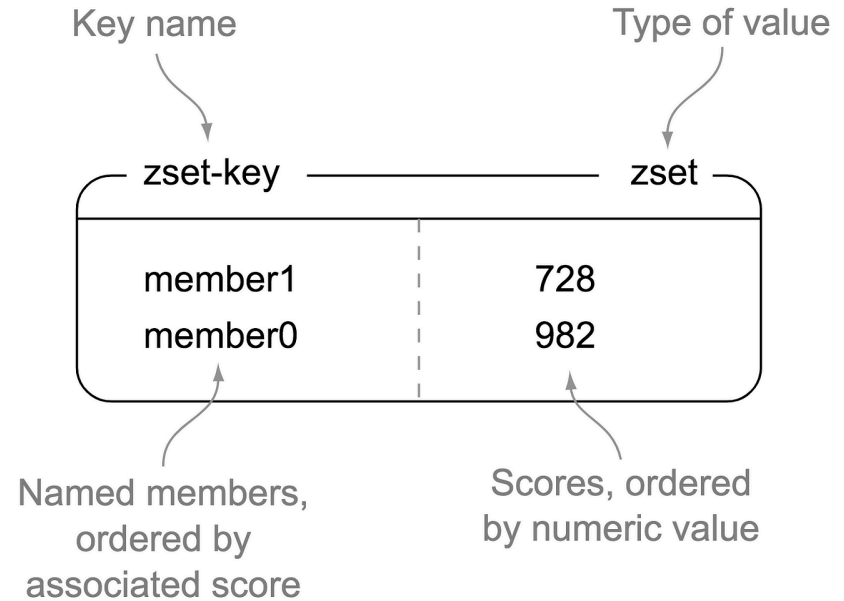


Valkey – Query Caching

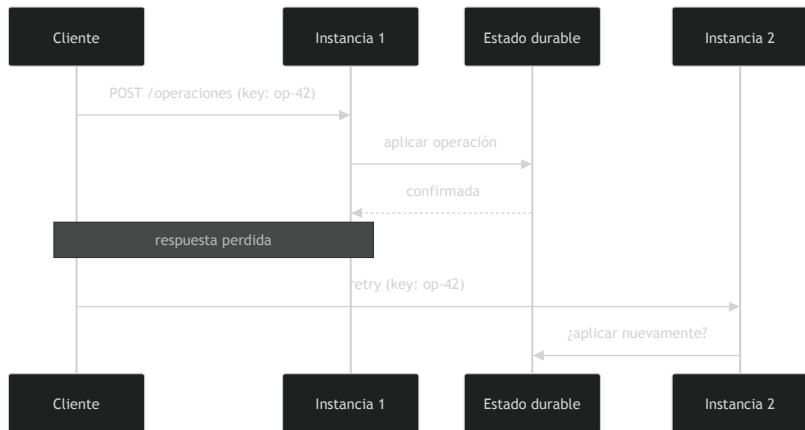


Valkey – Top N tables

- Con un Sorted Set se implementa una tabla de IDs y valores.
- La tabla se ordena automáticamente ante cada ingreso o modificación del valor.
- Es ideal para rankings, conteo de eventos distribuidos o procesamiento de datos por lotes.



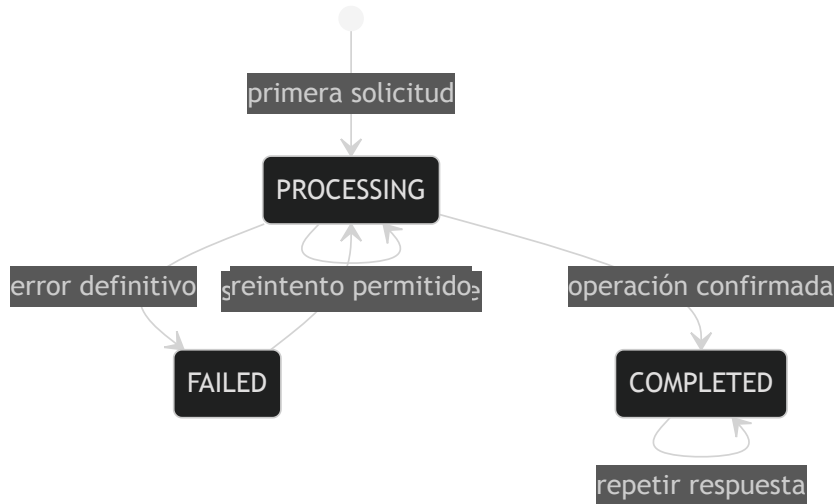
Origen de ejecuciones duplicadas



- Un timeout no informa si la operación falló o si se perdió la respuesta.
- Clientes, SDKs, gateways y servicios intermedios pueden reintentar.
- Un consumidor puede caer después del commit y antes del ACK.
- Un mensaje pendiente puede entregarse nuevamente durante la recuperación.

Fuente: <https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/>

Ciclo de vida de una clave de idempotencia



El registro puede incluir:

- identidad del cliente y de la operación;
- hash de los parámetros;
- estado de procesamiento y propietario;
- código y cuerpo de la respuesta;
- timestamps y política de expiración.

La misma key con parámetros diferentes debe rechazarse.

Ventanas de falla en la deduplicación

Registrar antes del efecto

1. SADD processed message:42
2. [caída del proceso]
3. aplicar cambio de negocio

El mensaje queda registrado, pero el cambio nunca se realiza. Las entregas posteriores se descartan.

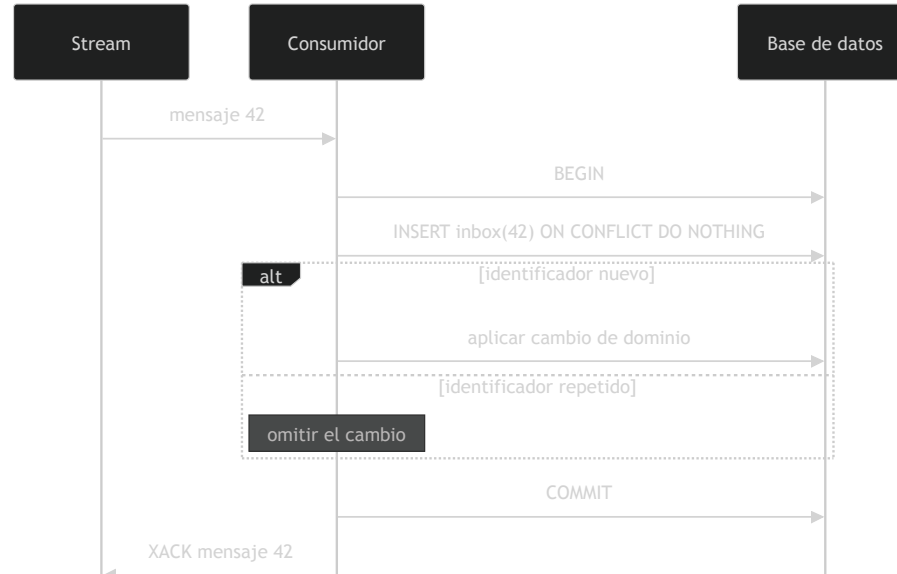
Registrar después del efecto

1. aplicar cambio de negocio
2. [caída del proceso]
3. SADD processed message:42

Una nueva entrega vuelve a aplicar el cambio.

Requisito: el registro de idempotencia y el efecto de negocio deben compartir una frontera atómica, o debe existir un protocolo de recuperación.

Inbox transaccional para consumidores



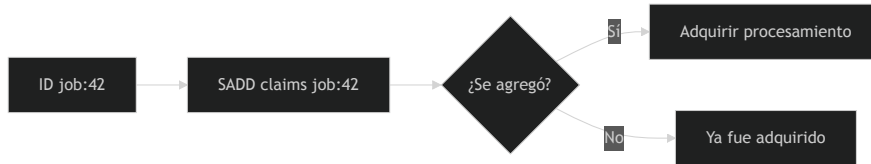
- `inbox.message_id` posee una restricción **UNIQUE**.
- El registro y el cambio de dominio se confirman en la misma transacción.
- Si el ACK se pierde, la siguiente entrega detecta el identificador repetido.

Fuente: <https://microservices.io/patterns/communication-style/idempotent-consumer.html>

Valkey – Idempotencia bajo concurrencia

- Con un Set se puede adquirir atómicamente el derecho a procesar un identificador.
- SADD devuelve 1 para el primer registro y 0 si el miembro ya existía.
- Esto evita la ejecución concurrente, pero no demuestra que el proceso haya finalizado.

```
valkey> SADD claims job:42
(integer) 1
valkey> SADD claims job:42
(integer) 0
valkey> SISMEMBER claims job:42
(integer) 1
```



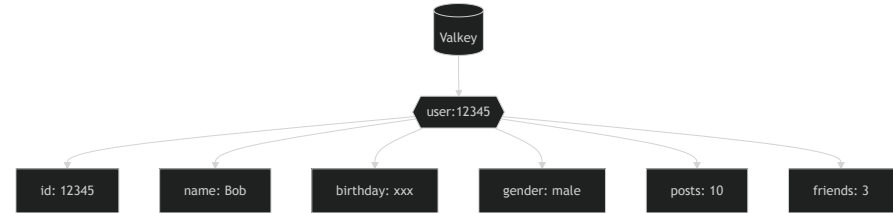
Valkey – Adquisición concurrente



La marca de adquisición requiere expiración, finalización y recuperación explícitas.

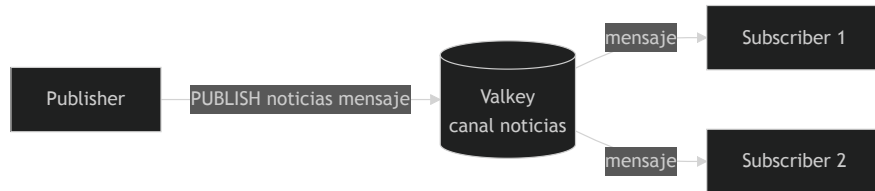
Valkey – Distributed Hash Map

- Permite realizar operaciones CRUD sobre registros en memoria.
- Los hashes asocian pares atributo-valor a una key.
- Los atributos se pueden establecer, incrementar, decrementar o borrar.
- Son ideales para almacenar sesiones con múltiples atributos.



Valkey – Pub/Sub

- Funciona como un típico Pub/Sub.
- Los canales no necesitan estar predefinidos: al publicar un mensaje, el canal queda disponible.
- Los suscriptores indican a qué canales se desean suscribir y pueden hacerlo a más de uno a la vez.
- Los mensajes no se persisten y no pueden consultarse después de su publicación.
- La entrega es *at-most-once*: si el suscriptor no recibe el mensaje, se pierde.



Semánticas de entrega de mensajes

At-most-once

- Cada mensaje se entrega una vez o ninguna.
- No existe reentrega frente a una falla.
- Evita duplicados, pero admite pérdida.
- Ejemplo: Valkey Pub/Sub.

At-least-once

- Se reintenta hasta obtener confirmación.
- Evita pérdida mientras el mensaje esté disponible.
- Admite entregas y ejecuciones repetidas.

Efecto idempotente

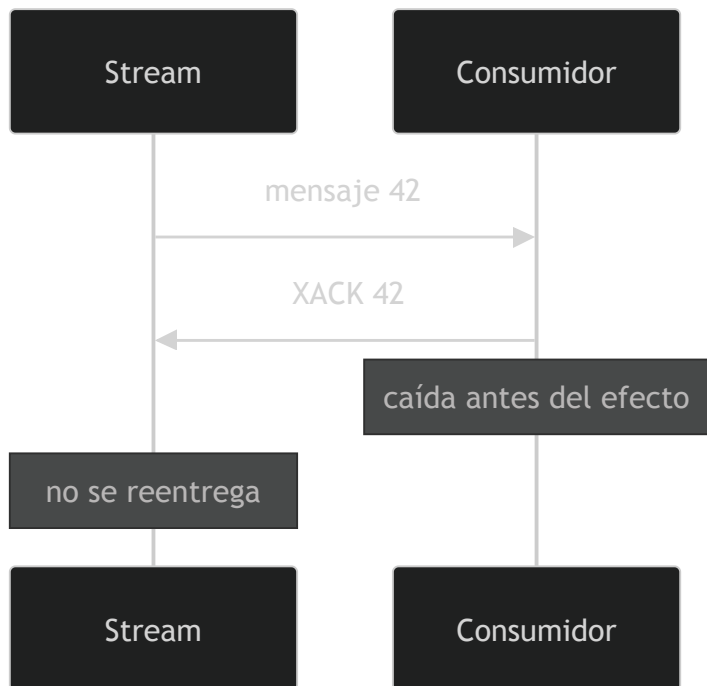
- La infraestructura puede entregar más de una vez.
- El consumidor reconoce la identidad del mensaje.
- El cambio de negocio se aplica una sola vez.

Exactly-once

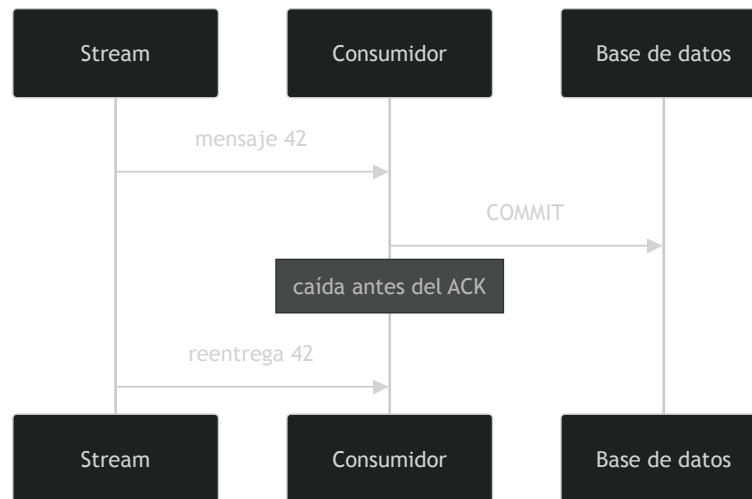
- Debe especificarse el alcance de la garantía.
- Una entrega única no garantiza un efecto externo único.
- Requiere coordinación entre consumo, estado y efectos.

Posición del ACK y ventanas de falla

ACK antes del efecto

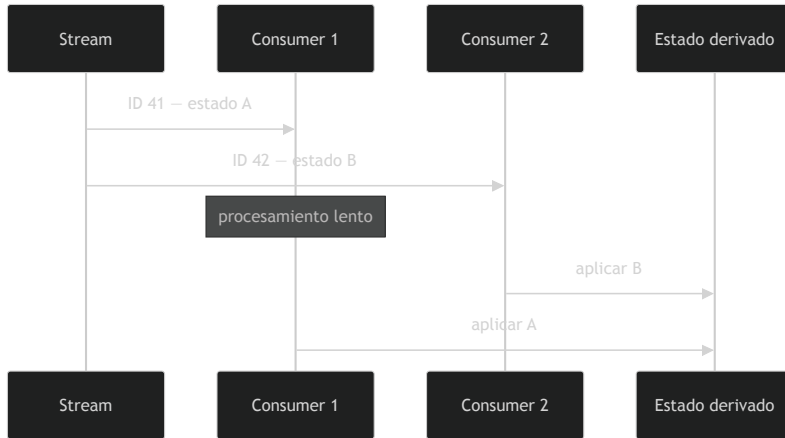


ACK después del efecto



El mensaje puede aplicar el efecto más de una vez si el consumidor no es idempotente.

Alcance del ordenamiento

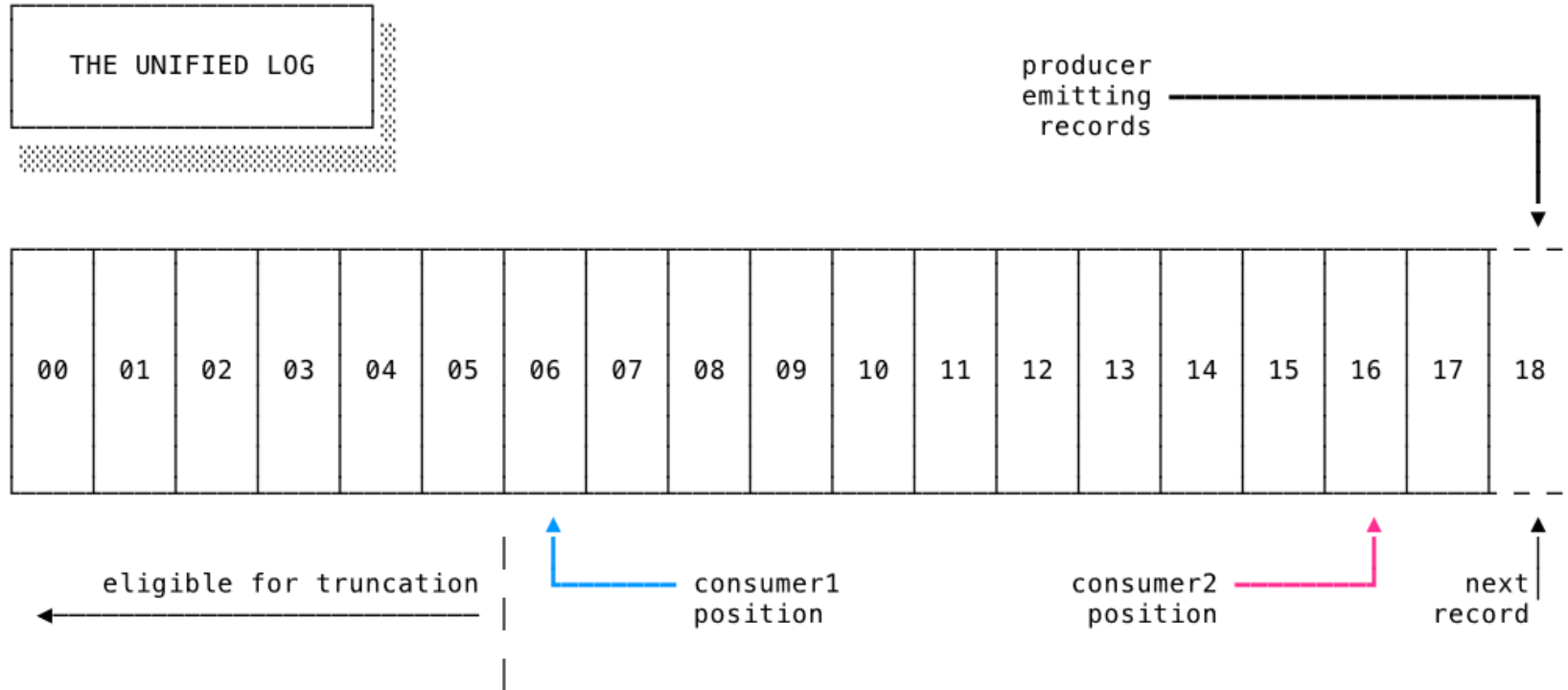


- El log conserva el orden de sus IDs.
- La distribución entre consumidores permite procesamiento paralelo.
- El orden de finalización puede diferir del orden de entrega.
- Productores independientes tampoco establecen automáticamente un orden causal.
- Si una entidad requiere orden, sus mensajes deben compartir una ruta serializada.

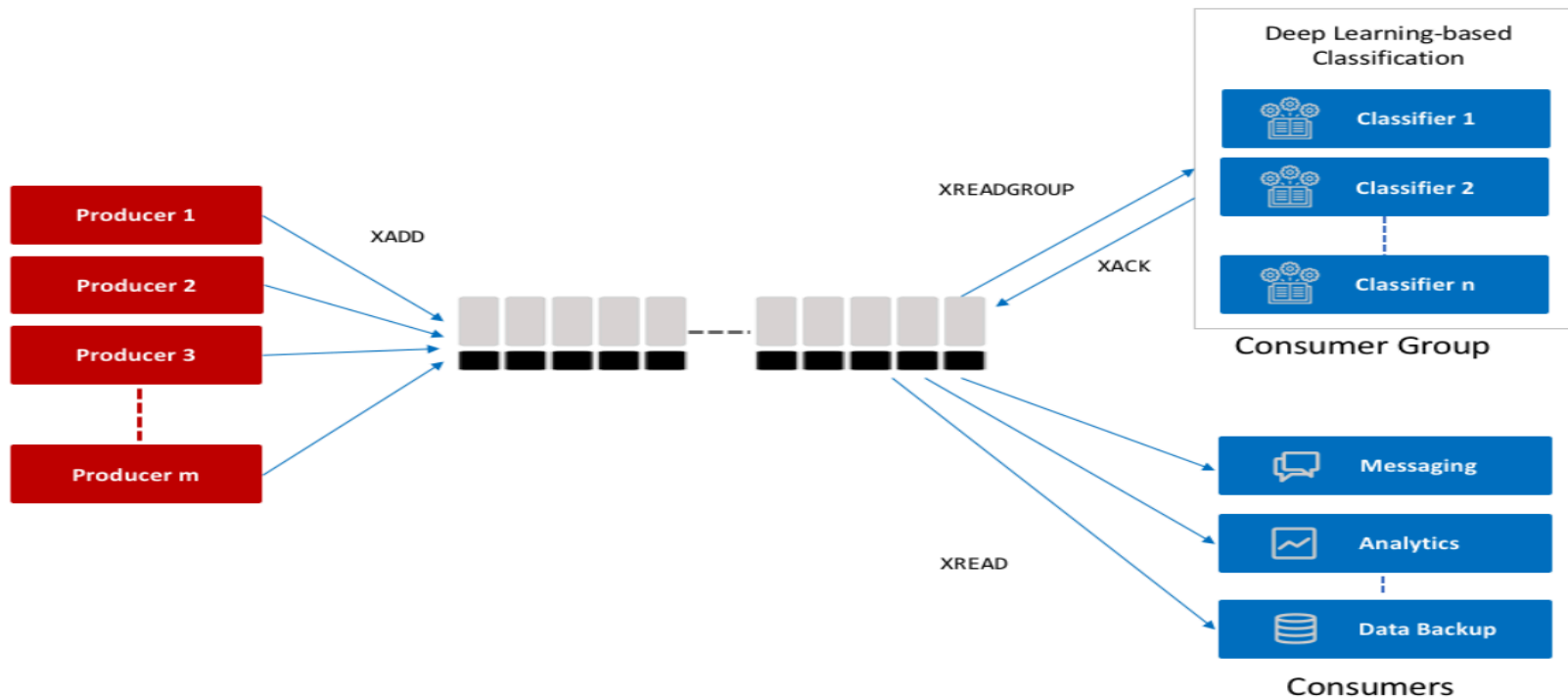
Valkey – Streaming

- Un stream es una secuencia ordenada de datos.
- El feeder agrega datos sólo al final de la secuencia.
- Los readers registran el último dato leído y consultan nuevos datos a partir de esa posición.
- En cualquier momento, el reader puede volver a leer desde el principio de la secuencia.
- Hay distintas estrategias para recortar el comienzo de la secuencia y liberar memoria.

Valkey – Streaming: log y truncado

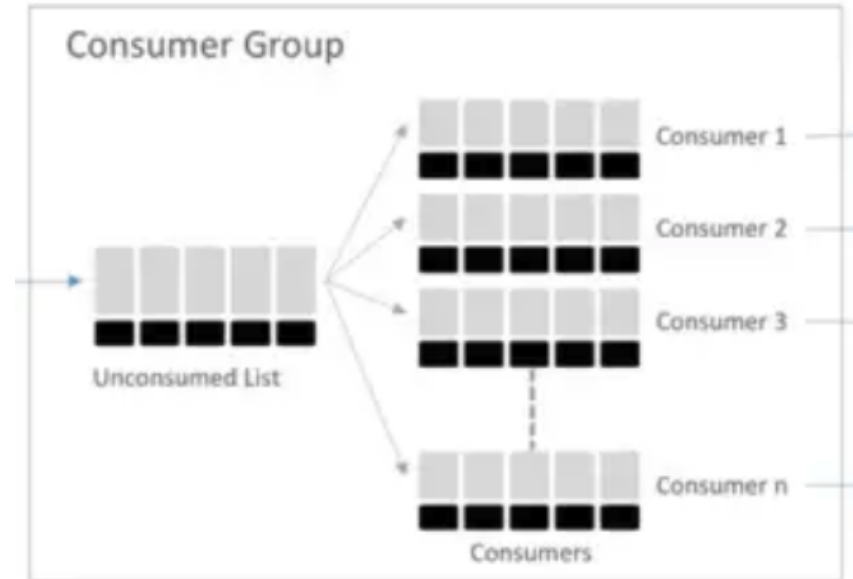


Valkey – Streaming: producers y consumer groups



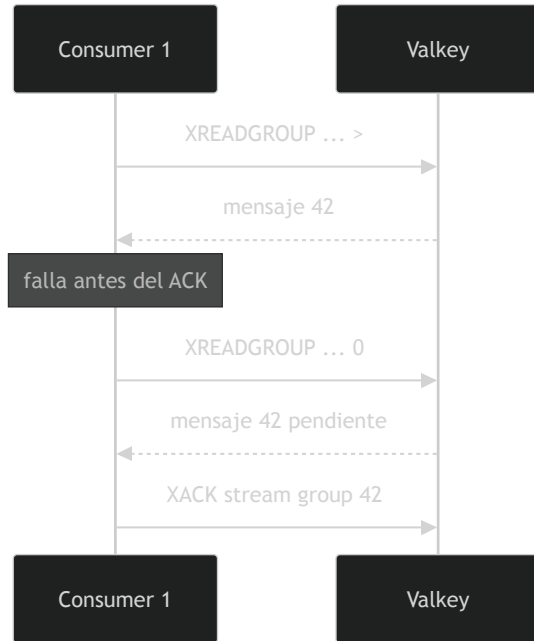
Valkey – Streaming: distribución y ACK

- Consumer Groups
 - Grupos de consumidores que pueden procesar indistintamente cualquier elemento del stream.
 - Cada consumer se identifica unívocamente.
- Valkey asegura que:
 - Cada elemento se envía a un solo consumidor dentro del grupo.
 - Siempre se envía el primer ID nunca consumido.
- Se espera un ACK del consumidor por cada mensaje enviado.

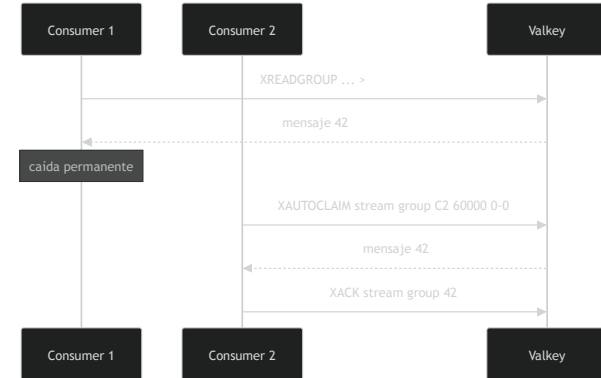


Valkey – Streaming Recovery/Claim

Recovery



Claim



Fuente: <https://valkey.io/topics/streams-intro/>